SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1417 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: Total Marks:50

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I Ragavi K (192524364)*(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Ragavi K*

Question 1: Trace Lexical and Syntax Analysis

Statement: result = (a + b) * c - 10;

1.1 Lexical analysis – lexemes and tokens

The scanner reads the character stream left to right, discards white space, and groups characters into the longest valid lexemes. Each lexeme is returned to the parser as a token.

#	Lexeme	Token	Category
1	result	ID	Identifier (operand)
2	=	ASSIGN_OP	Operator
3	(	LPAREN	Delimiter
4	a	ID	Identifier (operand)
5	+	ADD_OP	Operator
6	b	ID	Identifier (operand)
7	)	RPAREN	Delimiter
8	*	MUL_OP	Operator
9	c	ID	Identifier (operand)
10	-	SUB_OP	Operator
11	10	INT_CONST	Constant (operand)
12	;	SEMI	Delimiter

Token stream: <id,result> <=> <(> <id,a> <+> <id,b> <)> <*> <id,c> <-> <num,10> <;> (12 tokens). Identifiers are entered in the symbol table and the token carries a pointer to the entry.

Operators: =, +, *, - **Operands:** result, a, b, c, 10 **Delimiters:** ( ) ;

1.2 Syntax analysis – building the parse tree

The parser checks the token stream against the grammar of expressions. A grammar that already encodes precedence and associativity is used:

```
S -> id = E ;
E -> E + T | E - T | T        (lowest precedence, left-associative)
T -> T * F | T / F | F        (higher precedence, left-associative)
F -> ( E ) | id | num         (highest: parentheses and operands)
```

Leftmost derivation:

```
S => id = E ;
  => id = E - T ;
  => id = T - T ;
  => id = T * F - T ;
  => id = F * F - T ;
  => id = ( E ) * F - T ;
  => id = ( E + T ) * F - T ;
  => id = ( T + T ) * F - T ;
  => id = ( F + T ) * F - T ;
  => id = ( id + T ) * F - T ;
  => id = ( id + F ) * F - T ;
  => id = ( id + id ) * F - T ;
  => id = ( id + id ) * id - T ;
  => id = ( id + id ) * id - F ;
  => id = ( id + id ) * id - num ;
```

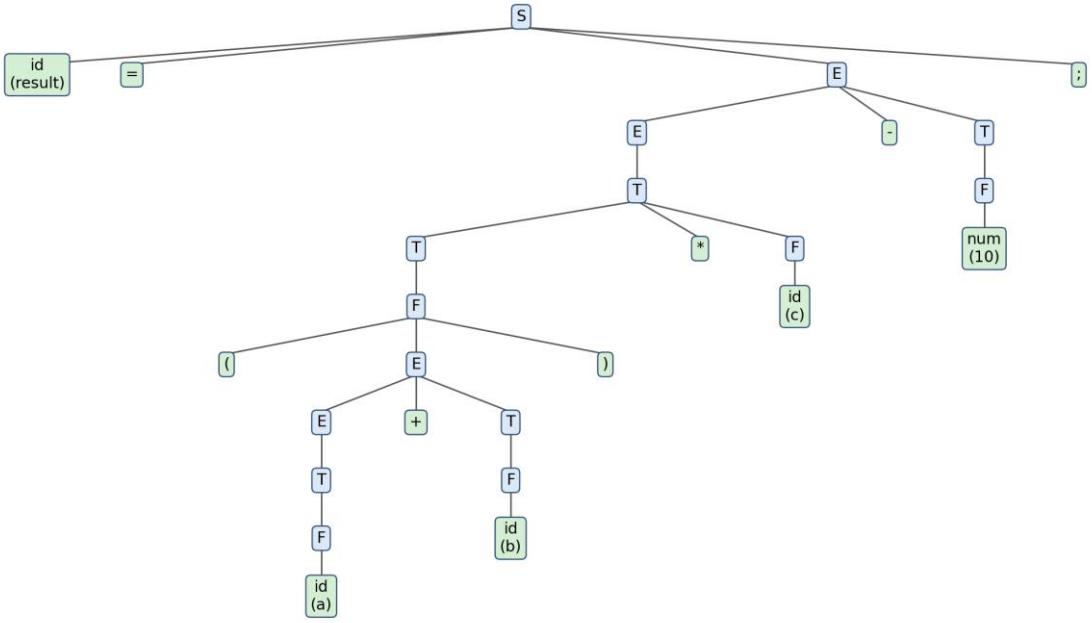


*Figure 1(a): Parse tree for result = (a + b) * c - 10;*

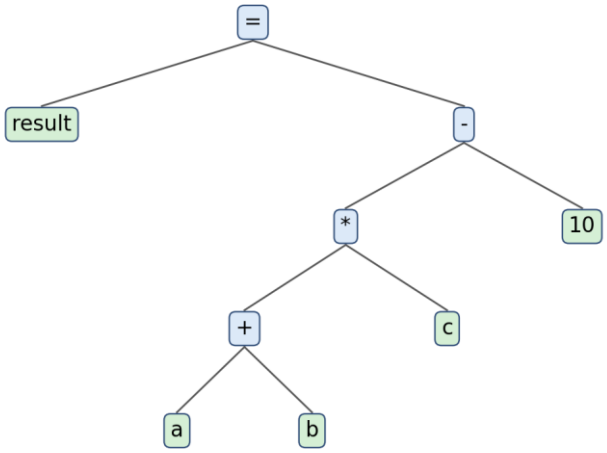


Figure 1(b): Abstract syntax tree (operators as internal nodes)

1.3 Precedence and associativity

- **Parentheses first:** (a + b) is derived inside F, the deepest grammar level, so it is evaluated before the * operator.
- **Precedence:** * and / are produced at level T, below + and - at level E. Therefore * binds tighter than -, giving ((a + b) * c) - 10 and not (a + b) * (c - 10).
- **Associativity:** the rules E -> E - T and T -> T * F are left-recursive, so equal-precedence operators group left to right (a - b - c = (a - b) - c). Assignment (=) is right-associative.
- **Evaluation order from the tree (bottom-up):** t1 = a + b; t2 = t1 * c; t3 = t2 - 10; result = t3.

1.4 Role of semantic analysis

- **Declaration check:** result, a, b, c must be declared; each name is looked up in the symbol table.
- **Type checking:** if a, b, c are int, then a + b is int, (a + b) * c is int, and the constant 10 is int, so the expression has type int. If a float were present, an implicit int-to-float conversion node would be inserted.
- **Assignment check:** the left side (result) must be a modifiable l-value whose type is compatible with the expression type.

- **Output:** an annotated syntax tree, passed on to intermediate code generation.

Question 2: Lexical Ambiguity and the Longest-Match Rule

Statement: value = a >= b && c != d;

2.1 How the statement is scanned

Lexeme	Token	Class
value	ID	Identifier
=	ASSIGN	Assignment operator
a	ID	Identifier
>=	RELOP (GE)	Relational operator
b	ID	Identifier
&&	AND	Logical operator
c	ID	Identifier
!=	RELOP (NE)	Relational operator
d	ID	Identifier
;	SEMI	Separator

Identifiers match letter (letter | digit)*. Operators are matched by their own patterns, and white space is skipped.

2.2 The ambiguity

- **> versus >=:** after reading ">" the scanner could stop and return GT, or continue because the next character "=" extends the lexeme to GE. Both are valid prefixes of the input.
- **! versus !=:** after "!" the scanner could return the logical NOT, or read "=" and return NE.

Splitting "a >= b" as ">" followed by "=" would give a wrong token stream and a false syntax error.

2.3 The longest-match (maximal munch) rule

When more than one pattern can match at the current position, the scanner chooses the pattern that matches the longest input. At ">=" the pattern ">=" matches 2 characters and ">" matches 1, so GE is selected. At "!=" the pattern "!=" (2 characters) beats "!" (1 character), so NE is selected. If the character after ">" were not "=", the scanner would retract the lookahead and return GT. The diagram below shows this behaviour.

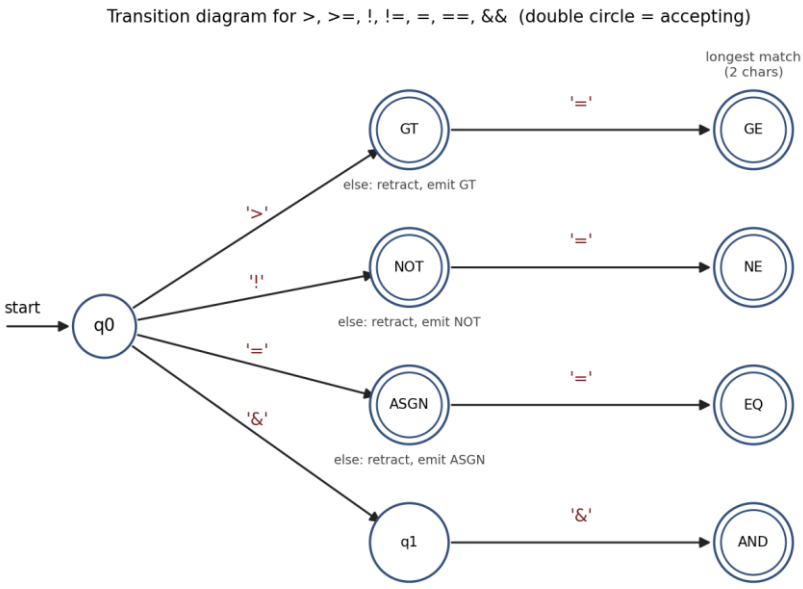


Figure 2: Transition diagram for relational, logical, and assignment operators

2.4 Token specification (LEX)

Token	Regular expression
Identifier	[A-Za-z_][A-Za-z0-9_]*
Assignment	=
Relational	< <= > >= == !=
Logical	&& || !
Separator	;

```
%{
#include <stdio.h>
%}
letter  [A-Za-z_]
digit   [0-9]
%%
[ \t\n]+                { /* skip white space */ }
"&&"                      { printf("AND\n"); }
"||"                     { printf("OR\n"); }
">="                      { printf("RELOP GE\n"); }
"<="                      { printf("RELOP LE\n"); }
"=="                      { printf("RELOP EQ\n"); }
"!="                      { printf("RELOP NE\n"); }
">"                       { printf("RELOP GT\n"); }
"<"                       { printf("RELOP LT\n"); }
"!"                      { printf("NOT\n"); }
"="                       { printf("ASSIGN\n"); }
{letter}({letter}|{digit})*  { printf("ID(%s)\n", yytext); }
";"                      { printf("SEMI\n"); }
.                       { printf("Lexical error: %s\n", yytext); }
%%
int yywrap(void){ return 1; }
int main(void){ yylex(); return 0; }
```

2.5 Justification of rule order

- LEX applies **longest match first**; only when two rules match the same length does it choose the **earlier rule**. So ">=" is chosen over ">" automatically, whatever their order.
- Multi-character operators (&&, >=, !=) are still listed **before** their single-character prefixes (>, !). This makes the intent clear and keeps the specification correct if the order is relied upon in other tools.
- Keywords (if, while, ...) must appear **before** the identifier rule, because they match the same length and the earlier rule wins.
- The catch-all rule "." must be **last**, or it would tie with single-character rules and hide valid operators. A single "&" matches only the catch-all and is reported as a lexical error.

Question 3: Token Stream and Lexical Error Handling

```
int total = 250;
float average = total / 5.0;
result@ = average + 10;
```

3.1 Line-by-line analysis

Line	Lexeme	Token	Classification
1	int	KEYWORD	Keyword
1	total	ID	Identifier
1	=	ASSIGN	Operator
1	250	INT_CONST	Constant
1	;	SEMI	Separator
2	float	KEYWORD	Keyword
2	average	ID	Identifier
2	=	ASSIGN	Operator
2	total	ID	Identifier
2	/	DIV_OP	Operator
2	5.0	FLOAT_CONST	Constant
2	;	SEMI	Separator
3	result	ID	Identifier
3	@	ERROR	Invalid character
3	=	ASSIGN	Operator
3	average	ID	Identifier
3	+	ADD_OP	Operator
3	10	INT_CONST	Constant
3	;	SEMI	Separator

3.2 The lexical error

The character **@** at line 3, column 7 does not begin any token pattern: it is not a letter, digit, operator, or separator. After the scanner returns the identifier "result" (longest match of the identifier pattern), the next character fails every rule, so the scanner reports an illegal character.

Note: "result" is also not declared in the fragment. That is a **semantic** error found later, not a lexical error.

3.3 Token stream before and after the error

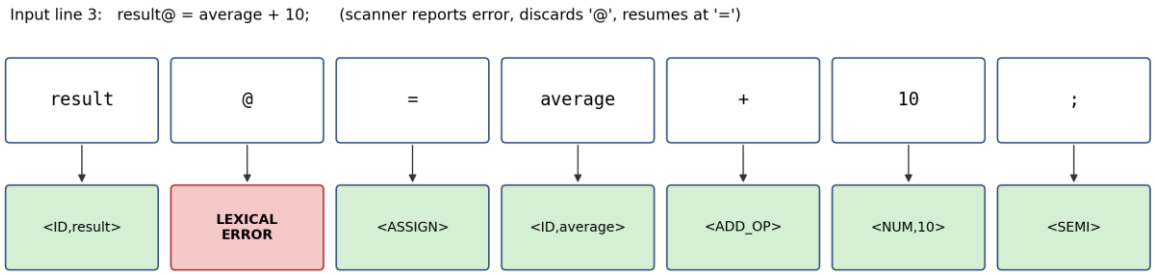


Figure 3: Scanning line 3 with error recovery

- **Up to the error:** <KEYWORD,int> <ID,total> <ASSIGN> <INT,250> <SEMI> <KEYWORD,float> <ID,average> <ASSIGN> <ID,total> <DIV> <FLOAT,5.0> <SEMI> <ID,result>
- **Error reported:** "Lexical error, line 3, column 7: illegal character '@'"
- **After the error:** <ASSIGN> <ID,average> <ADD_OP> <INT,10> <SEMI>

3.4 Error recovery

- **Panic-mode recovery (used here):** report the error with line and column, delete the offending character, and resume scanning from the next character. The scanner does not stop.
- **Other strategies:** insert a missing character, replace an incorrect character, or transpose two adjacent characters, choosing the fix that needs the fewest changes.
- **Benefit:** the rest of the program is still tokenised, so the parser and semantic analyzer can report further errors in a single compilation run. In LEX, this is the catch-all rule "." { error message }.

Question 4: Regular Expressions for Token Recognition

4.1 Regular expressions

Token category	Regular expression
KEYWORD	int | float | while | return
IDENTIFIER	letter (letter | digit)* where letter = [A-Za-z], digit = [0-9]
INT_CONST	digit+ = [0-9]+
FLOAT_CONST	digit+ . digit+ = [0-9]+\.[0-9]+
ASSIGN	=
RELOP	< | <= | > | >= | ==
ARITH_OP	+ | - | * | / = [+\-*/]
SEPARATOR	(|) | { | } | ; = [(){};]

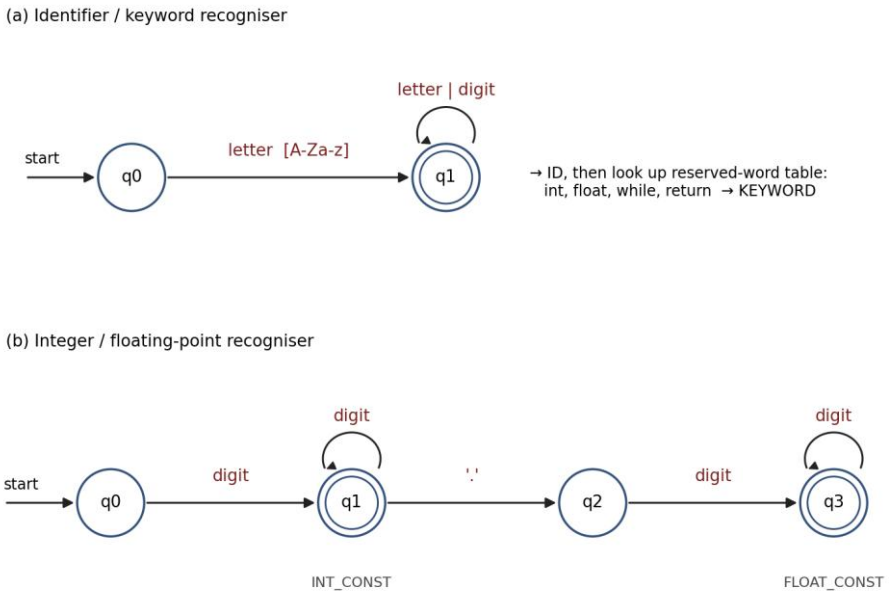


Figure 4: Finite automata for identifiers/keywords and numeric constants

4.2 Token stream

Line	Tokens
float total = price * 12.5;	<KEYWORD,float> <ID,total> <ASSIGN,=> <ID,price> <ARITH_OP,*> <FLOAT_CONST,12.5> <SEP,;>
while (total >= 100) {	<KEYWORD,while> <SEP,(> <ID,total> <RELOP,>=> <INT_CONST,100> <SEP,)> <SEP,{>
total = total - 10;	<ID,total> <ASSIGN,=> <ID,total> <ARITH_OP,-> <INT_CONST,10> <SEP,;>
}	<SEP,}>
return total;	<KEYWORD,return> <ID,total> <SEP,;>

Total tokens: 7 + 7 + 6 + 1 + 3 = **24**.

4.3 Keyword versus identifier

The word "float" matches the identifier pattern letter (letter | digit)*, so the identifier automaton accepts it. The scanner then looks up the lexeme in a reserved-word table (int, float, while, return). Since "float" is found, the token is KEYWORD; "total" and "price" are not found, so they stay ID. In LEX the same effect is obtained by listing the keyword rules before the identifier rule: both match the same length, so the earlier rule wins. A longer word such as "floating" is matched in full by the identifier rule (longest match) and is therefore an ID.

4.4 Recognising 12.5 as a floating-point constant

Starting at "1", the integer pattern [0-9]+ matches "12" (2 characters) and stops at ".". The floating pattern [0-9]+\.[0-9]+ continues through q1 --"."--> q2 --digit--> q3 and matches "12.5" (4 characters). By the longest-match rule the floating-point pattern wins, so 12.5 is returned as FLOAT_CONST and not as INT_CONST 12 followed by an error. The constant 100 has no ".", so only the integer pattern applies.

Question 5: Complete Compiler Processing

Statement: int average = total / count;

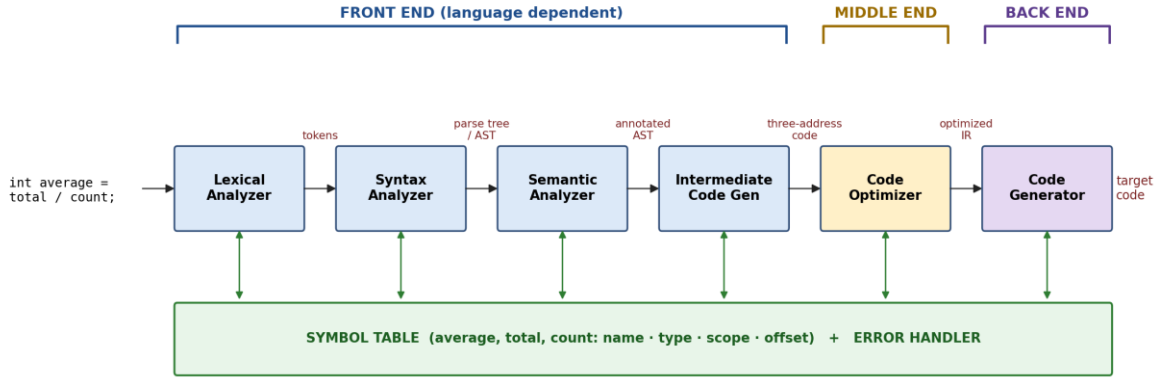


Figure 5: Compiler phases, symbol table, and front/middle/back end grouping

5.1 Phase-wise input and output

Phase	Input	Output
Lexical analysis	Character stream "int average = total / count;"	<KEYWORD,int> <ID,average> <ASSIGN,=> <ID,total> <DIV,/> <ID,count> <SEMI,;> (7 tokens)
Syntax analysis	Token stream	Parse tree / AST: Declaration(type=int, name=average, init=Divide(total, count))
Semantic analysis	AST + symbol table	Annotated AST: total and count declared, both int, total / count has type int, compatible with int average
Intermediate code	Annotated AST	Three-address code (below)
Optimization	Three-address code	Shorter equivalent IR (below)
Code generation	Optimized IR + symbol table	Target instructions (below)

5.2 Lexemes and tokens

Lexeme	int	average	=	total	/	count	;
Token	KEYWORD	ID	ASSIGN	ID	DIV	ID	SEMI

5.3 Syntax structure

```
Declaration
  |-- type : int
  |-- id   : average
  `-- init : Divide ( / )
         |-- id : total
         `-- id : count
```

5.4 Semantic analysis

- **Declaration check:** total and count must already be declared (symbol-table lookup); average is being declared here and must not already exist in the same scope.
- **Type check:** with total and count of type int, the integer division yields int, which matches the declared type of average. If either were float, the compiler would insert a float-to-int conversion (possible loss of precision, usually a warning).

- **Not detectable at compile time:** division by zero when count = 0 (a run-time error unless count is a known constant).

5.5 Intermediate representation and code generation

Three-address code:

```
t1 = total / count
average = t1
```

After optimization (copy propagation removes the temporary):

```
average = total / count
```

Target code (simple register machine):

```
LD   R1, total
LD   R2, count
DIV  R1, R1, R2
ST   average, R1
```

On x86 this corresponds to: mov eax,[total]; cdq; idiv dword [count]; mov [average],eax.

5.6 Role of the symbol table

Name	Type	Kind	Scope	Offset
total	int	variable	local	0
count	int	variable	local	4
average	int	variable	local	8

- **Lexical analysis:** installs each new identifier name.
- **Semantic analysis:** records the type and scope, and checks declarations and type compatibility.
- **IR generation / optimization:** refers to variables and temporaries by symbol-table entry.
- **Code generation:** reads the type, size, and memory offset to produce the correct load/store addresses.

5.7 Front end, middle end, and back end

Part	Phases	Role for this statement
Front end	Lexical, syntax, semantic analysis, IR generation	Depends on the source language. It validates "int average = total / count;" and converts it to language-independent IR.
Middle end	Machine-independent optimization	Improves the IR (removes t1) without knowing the source language or the machine.
Back end	Code generation, target-specific optimization, register allocation	Maps the IR to the target instructions (LD, DIV, ST or x86) and memory offsets.

Benefits of the division: each part can be developed, tested, and replaced independently. The IR is the common interface, so for M source languages and N target machines only M front ends and N back ends are needed instead of M x N complete compilers. Optimization is written once and shared by all languages and targets, and errors are localised to the phase that detects them.